

Abstracción de datos y tipos abstractos de datos

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Distinguir la interfaz de un tipo abstracto de datos de su implementación.
- Especificar un TAD con su dominio, las firmas de sus operaciones y ecuaciones, sin fijar la representación.
- Escribir código que dependa solo de la interfaz y sobreviva sin cambios a un cambio de representación.
- Enunciar el invariante de representación de una implementación y explicar quién lo establece y quién lo preserva.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA2: diseñar e implementar soluciones evaluando los costos de cada decisión de representación.
- OA3: describir ideas con argumentos precisos, aquí en forma de contratos y ecuaciones.

Referencia bibliográfica

R. Sebesta, *Concepts of Programming Languages*, 2015, capítulo 11 (tipos abstractos de datos y encapsulación). R. Thareja, *Data Structures Using C*, 2018, capítulo 2 (introducción a las estructuras de datos y los TAD).

Contenido I

- 1 Un tipo que el lenguaje no trae
- 2 Abstracción de datos
- 3 El área del programador
- 4 Dos representaciones
- 5 El contrato

Lo que dejó el paso de parámetros y la POO I

- Una función recibe, según la firma, una **copia**, una **dirección** o un **alias** de lo que le pasan.
- `private` cierra la puerta: los campos de un objeto solo cambian pasando por sus operaciones, y el compilador es el portero.
- Quedó una pregunta viva: aquella cuenta cuyo retiro no ocurría y no avisaba. Las operaciones validaban, pero en ningún lado estaba escrito *qué promete* cada una.

La pregunta de hoy

¿Dónde se escribe lo que una operación promete y lo que exige, de forma que el programa pueda apoyarse en eso y en nada más?

Contenido I

1 Un tipo que el lenguaje no trae

2 Abstracción de datos

3 El área del programador

4 Dos representaciones

5 El contrato

¿Cuánto vale un tercio? I

Antes de correrlo, prediga las tres líneas que imprime este programa.

```
#include <stdio>

int main() {
    double tercio = 1.0 / 3.0;
    double suma = 0.1 + 0.2;
    printf("%.17f\n", tercio);
    printf("%.17f\n", suma);
    if (suma == 0.3) {
        printf("0.1 + 0.2 es igual a 0.3\n");
    }
    if (suma != 0.3) {
        printf("0.1 + 0.2 no es igual a 0.3\n");
    }
    return 0;
}
```

La respuesta del computador I

Salida real

```
0.3333333333333331  
0.30000000000000004  
0.1 + 0.2 no es igual a 0.3
```

El problema

Un `double` guarda una aproximación binaria: ni un tercio ni un décimo caben exactos, y la comparación con `==` responde sobre las aproximaciones, no sobre los números.

- Si el programa necesita fracciones **exactas** —repartos, probabilidades, música, geometría— el tipo que hace falta es el racional: un cociente de dos enteros.

La respuesta del computador II

- C++ no lo trae. Hay que construirlo. La pregunta es *cómo* construirlo bien.

Primer intento: una estructura a la vista I

```
struct Racional {  
    int num;  
    int den;  
};
```

```
Racional crear(int p, int q) {  
    Racional r;  
    r.num = p;  
    r.den = q;  
    return r;  
}
```

$$\frac{1}{2} + \frac{3}{4} = \frac{4+6}{2 \times 4} = \frac{10}{8} = \frac{5}{4}$$

```
Racional sumar(Racional a, Racional b) {  
    Racional r;  
    r.num = a.num * b.den + a.den * b.num;  
    r.den = a.den * b.den;  
    return r;  
}
```

Primer intento: una estructura a la vista II

La suma usa la regla de las fracciones: $\frac{a}{b} + \frac{c}{d} = \frac{ad + cb}{bd}$.

El programa que lo usa I

```
int main() {  
    Racional a = crear(1, 6);  
    Racional b = crear(1, 3);  
    Racional s = sumar(a, b);  
    printf("%d/%d\n", s.num, s.den);  
    s.den = 0;  
    printf("%d/%d\n", s.num, s.den);  
    return 0;  
}
```

Salida real

```
9/18  
9/0
```

¿Quién tiene la culpa de ese 9/0?

Nadie violó ninguna regla del lenguaje I

El `main` compila sin una sola advertencia. La representación estaba a la vista y el programa la usó. Los problemas son dos, y son distintos:

Problema 1: el programa quedó soldado a la representación

Cada `printf("%d/%d", s.num, s.den)` conoce los campos. Si mañana la fracción se guarda de otra forma, **todas** esas líneas se rompen, estén donde estén.

Problema 2: el dominio quedó sin custodia

$9/0$ no es un número racional, pero la estructura lo almacena feliz. Ninguna operación posterior puede confiar en lo que recibe.

La solución a ambos problemas es la misma idea, y es el tema de hoy.

Contenido I

1 Un tipo que el lenguaje no trae

2 Abstracción de datos

3 El área del programador

4 Dos representaciones

5 El contrato

Definición I

Abstracción de datos (Sebesta, cap. 11)

La *abstracción de datos* separa las partes del programa que dependen de la **representación** de un dato de las partes que solo lo **usan**. Divide un tipo en dos:

- **Interfaz**: qué representa el dato, qué operaciones ofrece y qué promete cada una.
- **Implementación**: cómo se guarda el dato en memoria y el código de sus operaciones.

Definición II

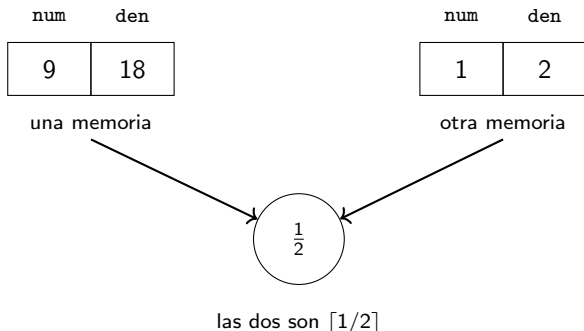
Tipo abstracto de datos (Thareja, cap. 2)

Un *tipo abstracto de datos* (TAD) es un conjunto de valores junto con sus operaciones y las propiedades de esas operaciones, especificado con precisión e independiente de cualquier implementación.

$\left\{ \begin{array}{l} \text{C++ 4 bytes} \\ \text{Python objeto} \end{array} \right.$

Una notación para hablar sin fijar la memoria I

Se escribe $\lceil x \rceil$ para decir “alguna representación del valor x , sin decir cuál”.



Dos memorias distintas pueden representar el mismo racional. La especificación habla del valor $\lceil p/q \rceil$; la memoria es asunto de la implementación.

La especificación del TAD Racional I

Dominio

$$\{p/q : p \in \mathbb{Z}, q \in \mathbb{Z}, q \neq 0\}$$

Interfaz: firmas

crear : Entero $\times$ Entero $\rightarrow$ Racional exige $q \neq 0$

numerador : Racional $\rightarrow$ Entero

denominador : Racional $\rightarrow$ Entero

La especificación del TAD Racional II

Interfaz: ecuaciones, para $r = \text{crear}(p, q)$ con $q \neq 0$

$$\text{denominador}(r) \neq 0$$

$$\text{numerador}(r) \cdot q = \text{denominador}(r) \cdot p$$

La segunda ecuación dice que el cociente se conserva. Fíjese en lo que **no** dice: no exige que $\text{numerador}(r)$ devuelva el p que llegó. Esa libertad es de la implementación, y hoy la vamos a usar.

El TAD en una caja I

TAD Racional
$\{p/q : p \in \mathbb{Z}, q \in \mathbb{Z}, q \neq 0\}$
crear : Entero $\times$ Entero $\rightarrow$ Racional, exige $q \neq 0$ numerador : Racional $\rightarrow$ Entero denominador : Racional $\rightarrow$ Entero

- Arriba, el nombre; en el medio, el conjunto de valores; abajo, las operaciones con sus firmas.
- Todo lo que el resto del programa puede saber del tipo está en esta caja. Nada más.

Las barreras de abstracción I

programas que usan racionales	
sumar, multiplicar, igual, imprimir	área del programador
crear, numerador, denominador	interfaz
int num, int den y su disciplina interna	implementación

- Cada capa habla **solo** con la de abajo.
- La aritmética se escribe con crear, numerador y denominador; jamás con num y den.
- Cambiar la capa de abajo no puede tocar a las de arriba. Esa es la promesa que el primer intento no podía hacer.

Contenido I

- 1 Un tipo que el lenguaje no trae
- 2 Abstracción de datos
- 3 El área del programador**
- 4 Dos representaciones
- 5 El contrato

La aritmética sobre la especificación I

Ecuaciones de las fracciones

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + cb}{bd}$$

$$\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}$$

¿Y la igualdad? $9/18$ y $1/2$ son el mismo racional con memorias distintas, así que comparar campo a campo no sirve desde afuera. Hace falta un criterio que funcione con *cualquier* representación.

La igualdad por productos cruzados I

Teorema

Sean $b \neq 0$ y $d \neq 0$. Entonces $\frac{a}{b} = \frac{c}{d}$ si y solo si $a \cdot d = c \cdot b$.

Demostración. Se procede de forma directa, probando las dos implicaciones.

Desarrollo.

- Si $\frac{a}{b} = \frac{c}{d}$, al multiplicar ambos lados por bd , que no es cero porque $b \neq 0$ y $d \neq 0$, queda $\frac{a}{b} \cdot bd = \frac{c}{d} \cdot bd$, es decir, $a \cdot d = c \cdot b$.
- Si $a \cdot d = c \cdot b$, al dividir ambos lados entre $bd \neq 0$ queda $\frac{ad}{bd} = \frac{cb}{bd}$, es decir, $\frac{a}{b} = \frac{c}{d}$.

La igualdad por productos cruzados II

Conclusión. Por lo tanto, se puede concluir que la igualdad de dos racionales se decide con dos productos de enteros, sin dividir y sin conocer la representación. ■

El área del programador en código I

Solo aparecen crear (el constructor), numerador y denominador.
Ninguna línea nombra num ni den.

```
Racional sumar(Racional a, Racional b) {  
    int p = a.numerador() * b.denominador()  
        + a.denominador() * b.numerador();  
    int q = a.denominador() * b.denominador();  
    Racional r(p, q);  
    return r;  
}  
  
Racional multiplicar(Racional a, Racional b) {  
    int p = a.numerador() * b.numerador();  
    int q = a.denominador() * b.denominador();  
    Racional r(p, q);  
    return r;  
}
```

El área del programador en código II

```
bool igual(Racional a, Racional b) {  
    int lado1 = a.numerador() * b.denominador();  
    int lado2 = a.denominador() * b.numerador();  
    return lado1 == lado2;  
}  
  
void imprimir(Racional a) {  
    printf("%d/%d\n", a.numerador(), a.denominador());  
}
```

Ejercicio en parejas: predecir la igualdad I

Con la interfaz de la caja y el teorema de los productos cruzados:

Prediga el resultado

```
Racional a = Racional(2, 8);  
Racional b = Racional(1, 4);  
printf("%d\n", igual(a, b));
```

$$\begin{array}{l} 2 \times 4 == 1 \times 8 \\ 8 == 8 \end{array}$$

¿Imprime 1 o 0? Escriba primero los dos productos que calcula igual.

La traza de la igualdad I

Traza de igual(a, b)

variable	se calcula con	valores	resultado
lado1	numerador(a) * denominador(b)	2 · 4	8
lado2	denominador(a) * numerador(b)	8 · 1	8
lado1 == lado2		8 = 8	verdadero

- Imprime 1: las memorias 2|8 y 1|4 difieren campo a campo y aun así el racional es el mismo.
- La comparación funciona aunque no sepamos cómo guarda la fracción la implementación. Está escrita contra la especificación.

Contenido I

- 1 Un tipo que el lenguaje no trae
- 2 Abstracción de datos
- 3 El área del programador
- 4 Dos representaciones**
- 5 El contrato

Versión 1: guardar la fracción tal como llega l

```
class Racional {  
    private:  
        int num;  
        int den;  
  
    public:  
        Racional(int p, int q) {  
            num = p;  
            den = q;  
        }  
  
        int numerador() {  
            return num;  
        }  
  
        int denominador() {  
            return den;  
        }  
};
```

Versión 1: guardar la fracción tal como llega II

- `private` cierra la puerta que el primer intento dejó abierta:
`s.den = 0` ya no compila.
- Las ecuaciones de la interfaz se cumplen de la manera más directa: se devuelve lo que llegó.

El puntero escondido: this I

Dentro de un método, escribir `num` es una abreviatura. La forma completa existe y se llama `this`:

```
int numerador() {  
    return this->num;  
}
```

Qué es `this`

En cada método, `this` es un puntero al objeto sobre el que corre el método; en los de `Racional` su tipo es `Racional *`. Un método es una función que recibe, además de sus parámetros, la **dirección** del objeto: el paso por dirección que ya conocemos, escondido en la sintaxis.

El puntero escondido: this II

Por qué importa en el diseño de un TAD

La caja pide `numerador` : Racional $\rightarrow$ Entero: una operación que recibe un racional. En C++ ese argumento no va entre paréntesis; viaja en `this` cuando se escribe `a.numerador()`. Las operaciones del TAD son un solo código compartido y los valores son muchos: `this` conecta la operación con el valor sobre el que actúa en cada llamada.

- Por eso `a.numerador()` y `b.numerador()` ejecutan el mismo código con resultados distintos: cambia la dirección que viaja en `this`.
- `this` marca además de qué lado de la barrera está el código: `this->num` dentro de la clase es la implementación tocando su propia representación; `a.num` afuera no compila.

El puntero escondido: this III

- Se vuelve necesario cuando un parámetro tapa a un campo con su nombre:

```
Racional(int num, int den) {  
    this->num = num;  
    this->den = den;  
}
```

A la izquierda de cada asignación está el campo del objeto; a la derecha, el parámetro. Sin `this` no habría forma de distinguirlos.

La corrida de la versión 1 I

```
int main() {  
    Racional a(1, 6);  
    Racional b(1, 3);  
    Racional s = sumar(a, b);  
    Racional m = multiplicar(a, b);  
    imprimir(s);  
    imprimir(m);  
    Racional medio(1, 2);  
    Racional dieciochoavo(1, 18);  
    if (igual(s, medio)) {  
        printf("la suma vale un medio\n");  
    }  
    if (igual(m, dieciochoavo)) {  
        printf("el producto vale un dieciochoavo\n");  
    }  
    return 0;  
}
```

La corrida de la versión 1 II

Salida real

```
9/18
1/18
la suma vale un medio
el producto vale un dieciochoavo
```

El programa *sabe* que $9/18$ vale un medio, pero imprime $9/18$.
¿Podemos lograr que imprima $1/2$ **sin tocar** el área del programador ni el `main`?

Versión 2: guardar siempre la forma más simple I

La idea: que *toda* Racional que exista esté simplificada. Eso es una promesa interna de la implementación, y se escribe como fórmula.

Invariante de representación de la versión 2

Todo objeto con campos (num, den) cumple:

$$I_1 : den > 0$$

$$I_2 : \text{mcd}(|num|, den) = 1$$

- De I_1 e I_2 sale el cero único: si $num = 0$, entonces $\text{mcd}(0, den) = den = 1$, y el cero siempre es $0/1$.
- **Quién lo establece:** el constructor, que llama a `simplificar()` antes de terminar.

Versión 2: guardar siempre la forma más simple II

- **Quién lo preserva:** todas las operaciones del área del programador construyen sus resultados con el constructor, así que ningún objeto se lo salta.

Ejercicio en parejas: escribir simplificar I

El método privado que sostiene el invariante

```
class Racional {  
    private:  
        int num;  
        int den;  
  
        void simplificar() {  
            // dejar num/den en forma simplificada  
        }  
  
    public:  
        Racional(int p, int q) {  
            num = p;  
            den = q;  
            simplificar();  
        }  
        // numerador() y denominador() igual que en la version 1  
};
```


Ejercicio en parejas: escribir simplificar II

Tres decisiones por tomar:

- ¿Dónde vive el signo, para que $1/-2$ y $-1/2$ terminen idénticos?
- ¿Cómo se calcula el máximo común divisor con un `while`?
- ¿Qué pasa con `num == 0`?

Una solución I

```
void simplificar() {  
    int a = num;  
    int b = den;  
    if (a < 0) {  
        a = -a;  
    }  
    if (b < 0) {  
        b = -b;  
    }  
    while (b != 0) {  
        int t = a % b;  
        a = b;  
        b = t;  
    }  
    num = num / a;  
    den = den / a;  
    if (den < 0) {  
        num = -num;  
        den = -den;  
    }  
}
```

$$25 \quad 18$$

$$\begin{array}{l} 25 \text{ mod } 18 = 10 \\ 18 \text{ mod } 10 = 8 \\ 10 \text{ mod } 8 = 2 \\ 8 \text{ mod } 2 = 0 \end{array}$$

$$27 \quad 18$$

$$\begin{array}{l} 27 \text{ mod } 18 = 9 \\ 18 \text{ mod } 9 = 0 \end{array}$$

Una solución II

- El `while` es el algoritmo de Euclides sobre los valores absolutos: al salir, `a` es el máximo común divisor.
- El último `if` muda el signo al numerador, y así $den > 0$ siempre.

La traza de Euclides con 9 y 18 l

Estado de a y b en cada vuelta

momento	a	b	t = a % b
al entrar	9	18	9 mód 18 = 9
tras la 1. ^a vuelta	18	9	18 mód 9 = 0
tras la 2. ^a vuelta	9	0	el while termina

- El máximo común divisor de 9 y 18 es 9.
- $num = 9/9 = 1$ y $den = 18/9 = 2$: la memoria queda $1|2$ y el invariante se cumple.
- Con signos: $crear(1, -2)$ queda $-1|2$ y $crear(-3, -6)$ queda $1|2$, verificado en la máquina.

La corrida de la versión 2, con el mismo programa I

El área del programador y el main son **los mismos archivos de texto** de la versión 1: cero líneas cambiadas afuera de la clase.

Las dos salidas, lado a lado

línea	versión 1	versión 2
suma impresa	9/18	1/2
producto impreso	1/18	1/18
¿la suma vale un medio?	sí	sí
¿el producto vale un dieciochoavo?	sí	sí

- Las respuestas del contrato (las igualdades) coinciden. Lo que cambió es la memoria, y la impresión la refleja.
- Esto es lo que compramos con la barrera: la representación cambió por completo y nadie arriba se enteró.

Dónde se paga cada decisión I

	versión 1	versión 2
crear	guarda tal cual	calcula el mcd
igual	dos productos	dos productos
números internos	crecen al operar	quedan pequeños
impresión	como se acumuló	simplificada

El riesgo de dejar crecer los números

En la versión 1, cada suma multiplica denominadores: tras pocas operaciones encadenadas los campos se acercan al tope del rango del `int` que estudiamos al mirar los tipos por dentro, y el desbordamiento produce racionales falsos sin ningún aviso. La versión 2 opera con números ya reducidos y aleja ese tope.

Dónde se paga cada decisión II

Mismo contrato, costos distintos. Elegir representación es una decisión de ingeniería, y la barrera permite cambiarla cuando la balanza se inclina.

Contenido I

- 1 Un tipo que el lenguaje no trae
- 2 Abstracción de datos
- 3 El área del programador
- 4 Dos representaciones
- 5 El contrato**

La precondition: crear con cero l

¿Qué pasa con `crear(1, 0)`?

La especificación lo dice desde la caja: `crear` exige $q \neq 0$. Eso es una *precondición*: una obligación de quien llama, no de quien implementa. Si el llamador la viola, ninguna ecuación promete nada.

- Aquí está la respuesta a la pregunta que dejó la cuenta cuyo retiro no avisaba: lo que una operación promete y lo que exige **se escribe en la especificación**, antes de escribir una sola línea de implementación.
- Con el contrato a la vista, cada quien sabe su parte: el llamador responde por $q \neq 0$; la implementación, por las ecuaciones.

La precondition: crear con cero II

- Las estructuras que siguen en el curso (lista, pila y cola) se van a definir exactamente así: primero su contrato, después sus implementaciones.

Vigilar la precondition con assert I

La precondition escrita solo en la especificación depende de que todos la lean. La biblioteca estándar permite además vigilarla en la frontera:

```
#include <cassert>

Racional(int p, int q) {
    assert(q != 0);
    num = p;
    den = q;
    simplificar();
}
```

Vigilar la precondition con assert II

Qué hace assert

Evalúa la condición. Si es verdadera, el programa sigue como si la línea no existiera. Si es falsa, el programa se detiene en ese punto y anuncia el archivo, la línea y la condición que falló.

Salida real con `Racional imposible(1, 0)` en el programa

```
1/2
rational: racional_precondicion.cpp:37: Racional::Racional(int, int):
↳ Assertion `q != 0' failed.
```

- Compare con el 9/0 que viajó callado en el primer intento, y con aquel retiro que no ocurría y no avisaba: la violación del contrato ahora suena, en el punto exacto donde ocurrió.

Vigilar la precondition con assert III

- Detener el programa parece severo. No lo es: después de un denominador cero no queda nada que la implementación pueda prometer, y seguir corriendo solo aleja el error de su causa.

Qué tanto exigir: dos contratos posibles I

La exigencia también se diseña. Para crear hay al menos dos contratos defendibles, y no producen la misma clase:

	exige $q \neq 0$	exige $q > 0$
crear(1, -2)	vale y queda $-1/2$	viola la precondition
el signo lo acomoda	simplificar	quien llama
simplificar	mcd y ajuste de signo	solo el mcd
assert	assert($q \neq 0$)	assert($q > 0$)

- Con $q \neq 0$ el llamador vive tranquilo y la implementación trabaja de más. Con $q > 0$ la implementación pierde el ajuste de signo y la carga se muda a cada punto del programa que construya racionales.
- Las dos opciones son contratos legítimos. Lo que daña el diseño es no decidir, o decidir y no escribirlo: la precondition elegida va en la caja del TAD y el assert la copia textual.

Ejercicio de cierre: especificar sin implementar I

En parejas, escriban la caja del **TAD Dinero** para montos en pesos y centavos:

- El **dominio**: ¿qué pares (*pesos, centavos*) son montos válidos?
- Las **firmas** de crear, pesos, centavos y sumar, con sus precondiciones.
- Dos **ecuaciones** que deba cumplir crear: una para el rango de los centavos y una que diga que el monto se conserva.

Pista: `crear(3, 250)` puede ser un llamado válido o una violación de precondición. Las dos decisiones son defendibles; el contrato que escriban debe tomar una y decirla. Si es violación, escriban también el `assert` que la vigila.

Propuestos para la casa I

- 1 Agregue restar al área del programador usando solo crear, numerador y denominador. Verifique que corre idéntica sobre las dos versiones de la clase.
- 2 ¿Puede la representación interna ser un solo double con el valor de la división? Use la ecuación $\text{numerador}(r) \cdot q = \text{denominador}(r) \cdot p$ y el experimento del tercio para argumentar por qué no.
- 3 Proponga una tercera representación: campos $(\text{signo}, \text{num}, \text{den})$ con $\text{num} \geq 0$ y $\text{den} > 0$. Escriba su invariante de representación y diga qué cambia en la clase y qué queda intacto afuera.

Propuestas para la casa: números sin techo I

El desbordamiento quedó señalado como el riesgo de dejar crecer los campos. La salida no es un entero más ancho: es otra representación detrás de una interfaz.

4 Implemente el **TAD Natural** con esta interfaz:

$\text{cero} : \rightarrow \text{Natural}$

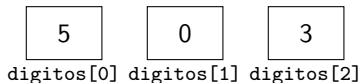
$\text{esCero} : \text{Natural} \rightarrow \text{Booleano}$

$\text{sucesor} : \text{Natural} \rightarrow \text{Natural}$

$\text{predecesor} : \text{Natural} \rightarrow \text{Natural}$, exige esCero falso

y la representación de un arreglo de dígitos en base 10, el menos significativo de primero:

Propuestas para la casa: números sin techo II



el 305, del dígito menos significativo al más significativo

sucesor suma uno al primer dígito y propaga el acarreo con un `while`; predecesor pide prestado en la otra dirección. Escriba el invariante de representación: el rango de cada dígito y qué pasa con los ceros a la izquierda. Con esta representación el techo del `int` desaparece: el número crece dígito a dígito.

Propuestos para la casa: números sin techo III

- 5 Sobre ese *Natural*, implemente el **TAD Entero** como pareja (*signo, magnitud*) con la misma interfaz. Dos cosas para observar al escribirlo: el invariante que hace único al cero, y que *predecesor* pierde su precondition, porque en $\mathbb{Z}$ todo número tiene predecesor. Un contrato puede volverse más generoso al cambiar el dominio.

Resumen I

- Un TAD es un conjunto de valores con operaciones y propiedades, especificado sin fijar la representación.
- La interfaz se define con firmas y ecuaciones; la notación $\lceil x \rceil$ permite hablar del valor sin hablar de la memoria.
- El área del programador se escribe solo contra la interfaz: por eso sobrevivió, línea por línea, al cambio de representación.
- El invariante de representación es una fórmula que la implementación se promete a sí misma; el constructor lo establece y las operaciones lo preservan.
- Las precondiciones reparten responsabilidades: el contrato dice qué exige y qué promete cada operación, y `assert` lo vigila en la frontera.

Resumen II

El tema que sigue

Los TAD Lista, Pila y Cola: sus contratos y sus ejemplos de uso.
Vale la pena llegar habiendo pensado qué operaciones esperaríamos de una pila de platos y de una fila de banco.

Referencias I

-  R. Sebesta. *Concepts of Programming Languages*. Pearson, 2015. Capítulo 11: tipos abstractos de datos y encapsulación.
-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 2: introducción a las estructuras de datos y los TAD.
-  D. Friedman y M. Wand. *Essentials of Programming Languages*. MIT Press, 2008. Capítulo 2: especificación de datos mediante interfaces.
-  H. Abelson y G. Sussman. *Structure and Interpretation of Computer Programs*. MIT Press, 1996. Sección 2.1: el racional como dato compuesto.